Faculty of Engineering
Mechatronics Engineering
Department



Republic of Yemen
Ministry of Higher Education
and Scientific Research
University of Sana'a

{ Computer Programming 2 }

Project Report: Thermal-Aware Path Planner

Done by

Abdulmlek Marwan Abdulmlek Al-sharee

**Academic. No. **

202574185

Supervised by

Eng. Asma Al-Ariqi

Contents

1. Introduction 1

2. Project Idea and Purpose 1

3. Problem Statement 2

4. Objectives 2

5. Main Features 3

6. System / Program Design 4

6.1 The C++ Engine 5

6.2 The MATLAB Visualization Application 6

6.3 SolidWorks Mechanical and Thermal Model 7

7. C++ / OOP Concepts Used 11

8. Explanation of the Most Important Parts of the Implementation 12

8.1 Point3D and the Distance Metric 12

8.2 Building the Graph from Irregular Simulation Data 12

8.3 The CostFunction Hierarchy 12

8.4 The A* Search Loop 13

8.5 Reading Real SolidWorks Thermal Data 13

8.6 The MATLAB Application's Callback Flow 13

9. Screenshots and Sample Outputs 14

10. Challenges Faced and How They Were Solved 17

11. Future Work 18

12. Conclusion 19

1. Introduction

Autonomous mobile robots are increasingly deployed in industrial facilities to carry out inspection, transport, and monitoring tasks. Most path-planning systems used in these robots are designed to answer a single question: what is the shortest collision-free route from a start point to a goal? This is typically solved by discretizing the environment into a grid or graph and applying a search algorithm such as A*.

Many real industrial environments, however, are not thermally uniform. Furnaces, ovens, exhaust ducts, and heavy machinery create localized zones of elevated temperature that a purely geometric planner cannot recognize, because “hot” is not the same thing as “blocked.” A robot that always takes the shortest path may repeatedly be routed close to equipment that degrades its battery, electronics, or mechanical components over time.

This project, the Thermal-Aware Path Planner, addresses that gap. It is an object-oriented C++ application that reads real temperature-field data exported from a SolidWorks thermal simulation, builds a navigable graph over that data, and searches for a path using the A* algorithm driven by a pluggable, temperature-sensitive cost function. The result is a system that can trade a small increase in travel distance for a large reduction in thermal exposure — and, because the cost function is decoupled from the search algorithm through polymorphism, the same engine can just as easily fall back to ordinary shortest-path behavior.

2. Project Idea and Purpose

The central idea of the project is that path planning for an industrial robot should not treat distance as the only cost. Consider two possible routes between a start and a goal position: Path A is short but passes directly next to a heat source, while Path B is somewhat longer but stays well away from it. A conventional planner will always choose Path A. The Thermal-Aware Path Planner is designed to recognize that Path B may, in fact, be the better choice — because the cost of a move is defined as a combination of the physical distance travelled and the thermal risk incurred by passing through a high-temperature node.

The project delivers two complementary pieces toward this idea. The first is a C++ engine — the core deliverable for this course — that loads a real 3D thermal dataset exported from a SolidWorks thermal study and computes both a conventional and a thermal-aware path between the same two points for direct comparison. The second

is an interactive MATLAB App Designer application that lets a user load a 2D thermal map, set start/goal points and cost weights through a graphical interface, and see the resulting path drawn directly over the thermal field. The two currently operate as independent implementations of the same underlying idea rather than one calling the other; Section 6 describes how they relate, and Section 11 discusses unifying them.

3. Problem Statement

Traditional path-planning systems represent the environment as a binary map of free space and obstacles, and search for the shortest path that avoids the obstacles. This model is insufficient in industrial settings with hazardous thermal zones, for two reasons: a high-temperature area is not necessarily an obstacle — it may be safe to cross briefly but risky to linger in — and distance-based planners have no mechanism to represent this kind of "soft" risk at all, since their cost function is purely geometric.

The problem this project addresses can therefore be stated as follows: given a 3D temperature field measured or simulated over an industrial environment, how can a mobile robot's path-planning algorithm be extended to produce a path that keeps peak thermal exposure low, while still reaching its destination through a reasonably efficient route, and without requiring the search algorithm itself to be redesigned for every new planning criterion?

4. Objectives

- Design an object-oriented C++ architecture that separates spatial representation (Point3D, Obstacle, Environment), graph representation (GraphNode, Graph), and search logic (AStar) into independent, reusable modules.
- Implement a pluggable cost-function interface (CostFunction) so new planning criteria — distance, temperature, or in principle any other cost — can be added without modifying the A* search algorithm.
- Correctly ingest a real thermal dataset exported from a SolidWorks thermal study, including unit conversion from Kelvin to Celsius and defensive handling of malformed data rows.
- Build a navigable graph automatically from an irregular, simulation-derived point cloud, rather than assuming a perfectly uniform grid.

- Demonstrate quantitatively, using the real dataset, that the thermal-aware cost function produces a materially safer path than a distance-only planner.
- Export the computed path to CSV for downstream analysis or visualization.

5. Main Features

- 3D geometric primitives — a Point3D class with full arithmetic and comparison operator support.
- Bounding-box obstacles — axis-aligned obstacle regions that the environment can test any point against.
- Automatic, irregular-grid graph construction — the Graph class builds itself directly from whatever thermal data points are supplied, using a custom nearest-neighbor search to determine connectivity along the six principal directions, rather than assuming fixed grid spacing.
- Swappable cost functions — DistanceCost and ThermalCost both implement the same CostFunction interface, so the A* planner can be switched between “shortest path” and “coolest path” behavior with a single line of setup code.
- Thermal-aware A* search — a full implementation of the A* algorithm (open/closed lists, g/h/f costs, path reconstruction) driven by whichever cost function is currently installed.
- Real SolidWorks data ingestion — ThermalDataReader parses the actual multi-column CSV export format produced by a SolidWorks thermal study and converts it into the engine's internal representation.
- CSV path export — CSVWriter saves the final path to a CSV file, automatically creating the destination folder if it does not already exist.
- Interactive MATLAB front-end — a MATLAB App Designer application (Thermal_Aware_planner) provides a graphical interface for loading a thermal map, setting start/goal points and cost weights, and visualizing the resulting path and planning metrics live.

6. System / Program Design

The completed project comprises two independent software artifacts built around the same core idea: choosing a path that balances distance against thermal exposure. Figure 1 shows both tracks. Section 6.1 describes the C++ engine, the primary deliverable for this course; Section 6.2 describes the MATLAB application, which provides an interactive graphical alternative driven by its own planning logic.

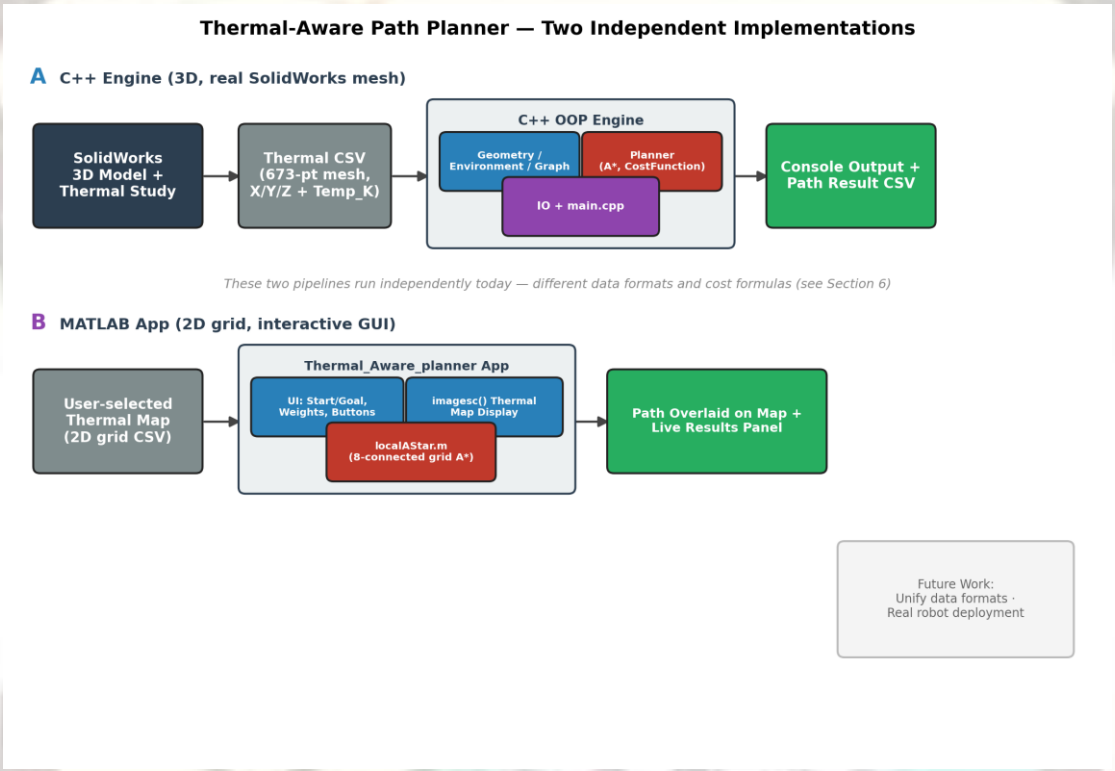


Figure 1. Two independent implementations: (A) the C++ engine driven by a real SolidWorks thermal export, and (B) the MATLAB App Designer application driven by a user-selected 2D grid.

6.1 The C++ Engine

The C++ engine follows a modular, single-responsibility folder structure. Each folder maps directly to one stage of Track A in Figure 1:

Module	Classes	Responsibility
Geometry	Point3D	3D coordinates, distance calculation, arithmetic and comparison operators
Environment	Environment, Obstacle	Bounding-box obstacles and environment-level “is this position blocked?” queries
Thermal	ThermalNode, ThermalMap	A single thermal sample point (position + temperature) and a collection of them
Graph	GraphNode, Graph	Search-graph representation built from the ThermalMap, plus custom nearest-neighbor connectivity
Planner	CostFunction, DistanceCost, ThermalCost, AStar	The cost-function interface, its two implementations, and the A* search itself
IO	CSVReader, CSVWriter, ThermalDataReader	Reading thermal data (two supported formats) and writing the computed path

main.cpp orchestrates a single run in twelve explicit steps: load the thermal data, build the Environment, add obstacles, build the Graph, define the start/goal points, construct an AStar planner, install the desired cost function, run the search, display the result, and export it to CSV.

6.2 The MATLAB Visualization Application

Thermal_Aware_planner.mlapp is a MATLAB App Designer application — itself implemented as a MATLAB class inheriting from matlab.apps.AppBase, echoing the same object-oriented principles as the C++ side in a different language. Its interface (Figure 3, Section 9) has three panels: an Environment panel for start/goal coordinates and loading a thermal map, a central axes that displays the map as a 2D image, and a Path Planner panel for setting distance/thermal weights and running the search.

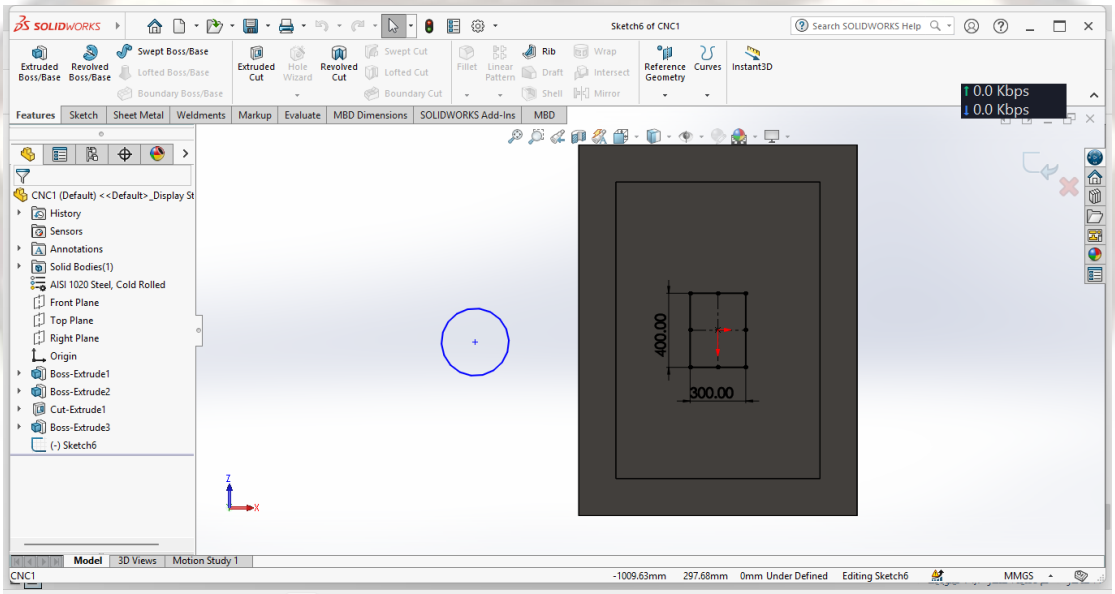
Component	Type	Responsibility
Thermal_Aware_planner	App class (AppBase)	Owns the UI and wires all three button callbacks
LoadMapButtonPushed	Callback	Opens a file picker, reads a 2D numeric grid via readmatrix, displays it with imagesc
PlanOptimalPathButtonPushed	Callback	Converts physical X/Y coordinates to grid indices, calls localAStar, plots the path, updates the results panel
ResetButtonPushed	Callback	Clears the map, plot, and results back to their default state
localAStar.m	Standalone function	An independent 2D, 8-connected A* implementation with a min-max normalized thermal cost

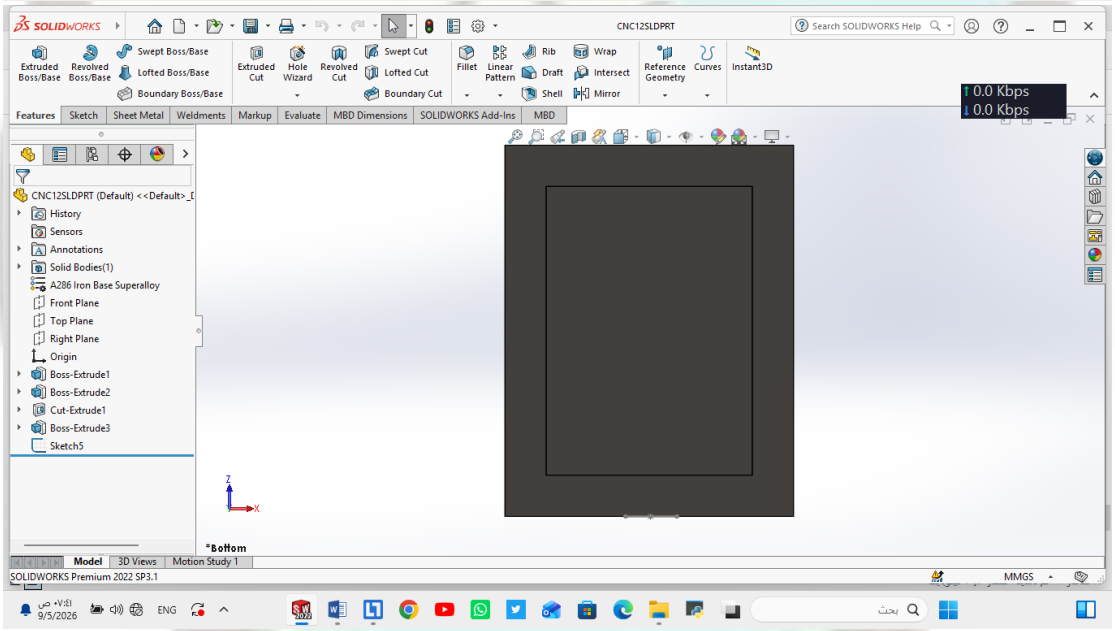
It is worth being precise about how this differs from the C++ engine. localAStar.m operates on a regular 2D matrix rather than an irregular 3D point cloud, considers eight neighbors (including diagonals) rather than six, and defines its thermal penalty as a min-max normalized value — (temperature − minTemp) / (maxTemp − minTemp) — rather than the C++ engine's fixed-threshold excess above a safe temperature. Both are legitimate ways to express “avoid heat,” but they are not numerically interchangeable, and the two engines were not designed to produce identical paths from the same input.

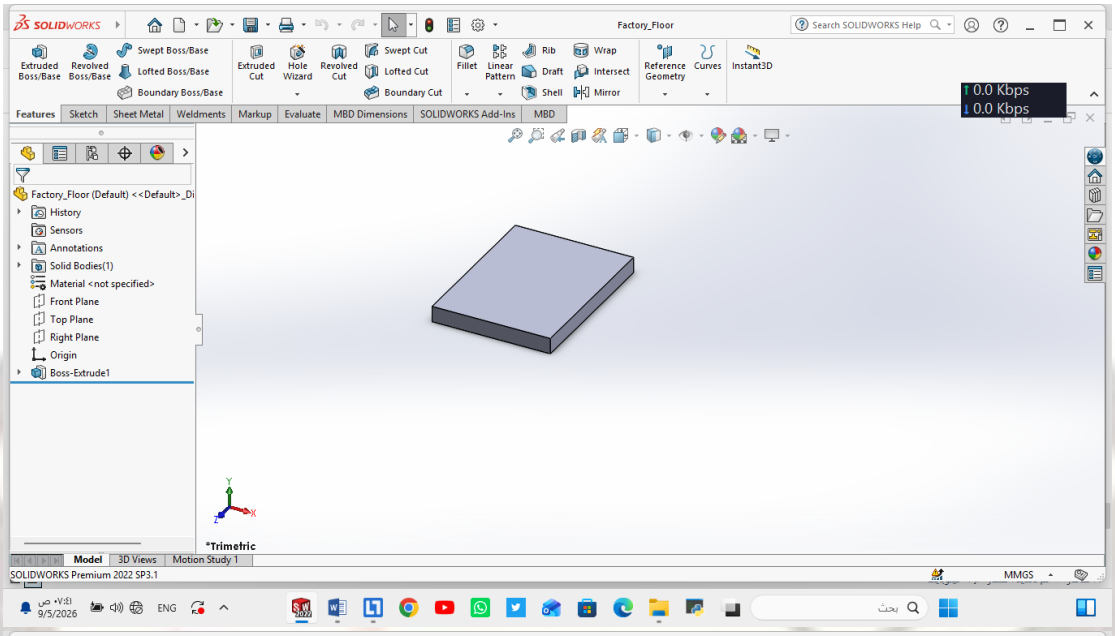
One detail is worth double-checking before quoting the results panel's numbers as the optimization objective: the search itself minimizes normalized thermal exposure, but calculatePathCost (used only to report the final ThermalCost and TotalCost figures shown to the user) sums the raw, non-normalized temperature values along the path instead. The application runs and displays a value either way, but that

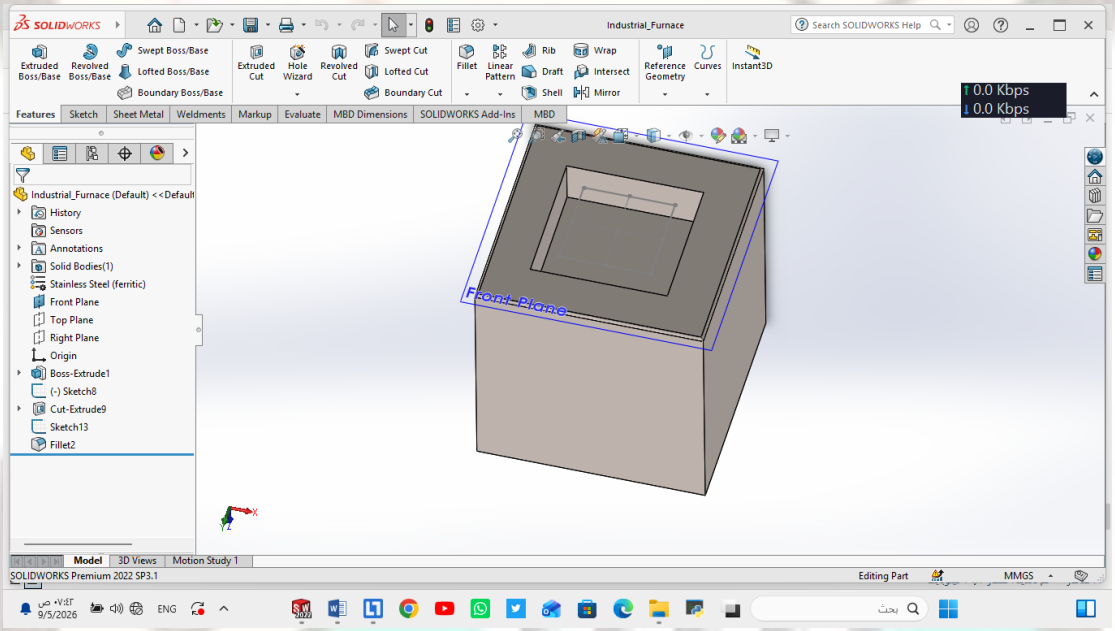
displayed number is not on the same scale as the cost the search actually optimized — see Section 11 for a suggested fix.

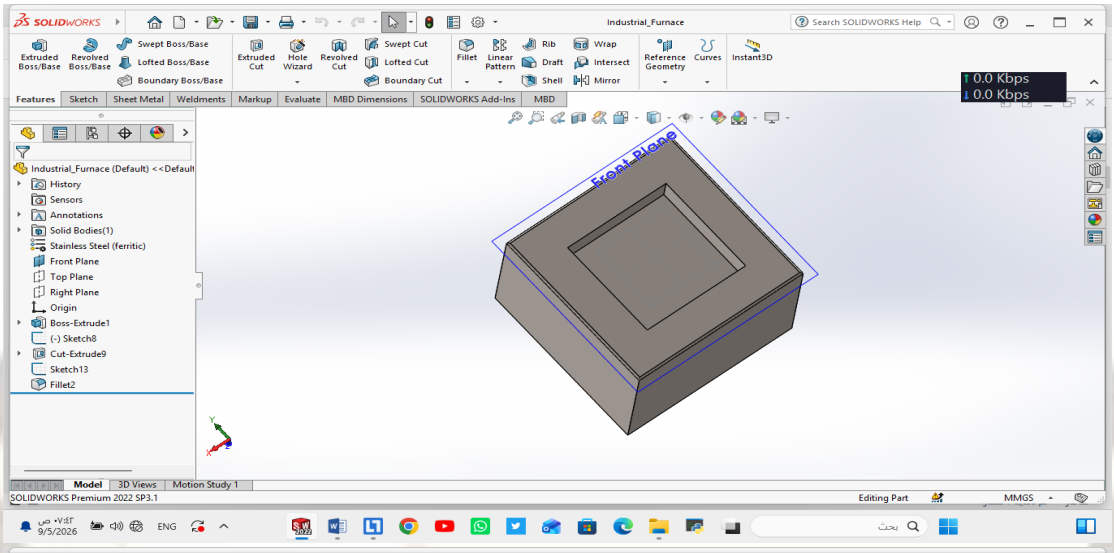
6.3 SolidWorks Mechanical and Thermal Model

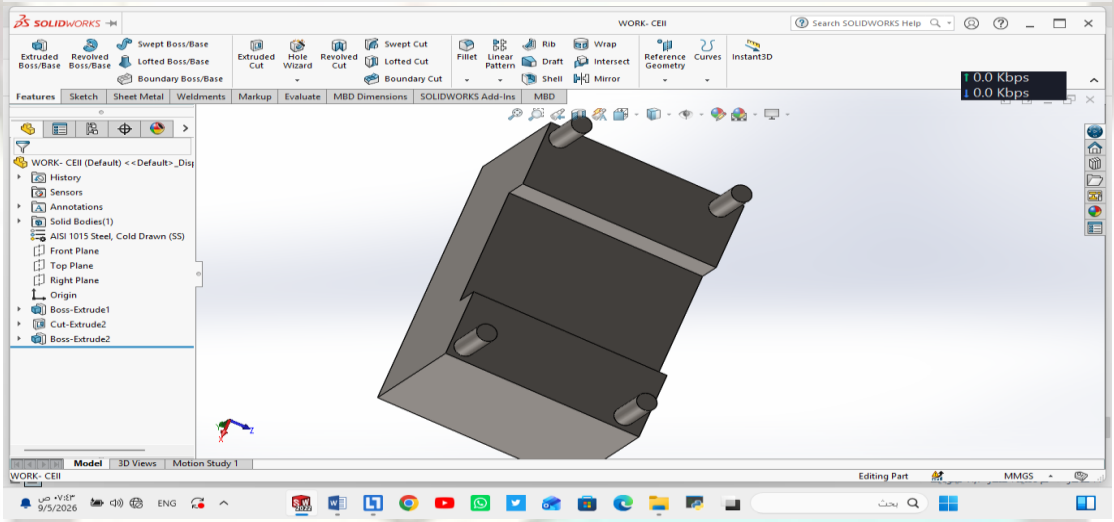


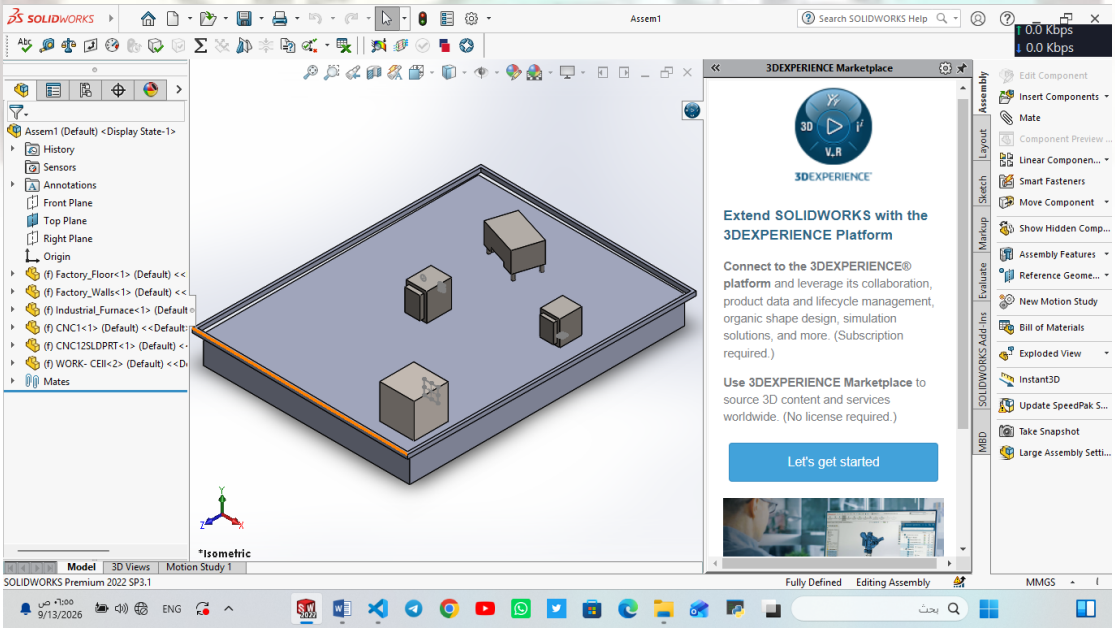


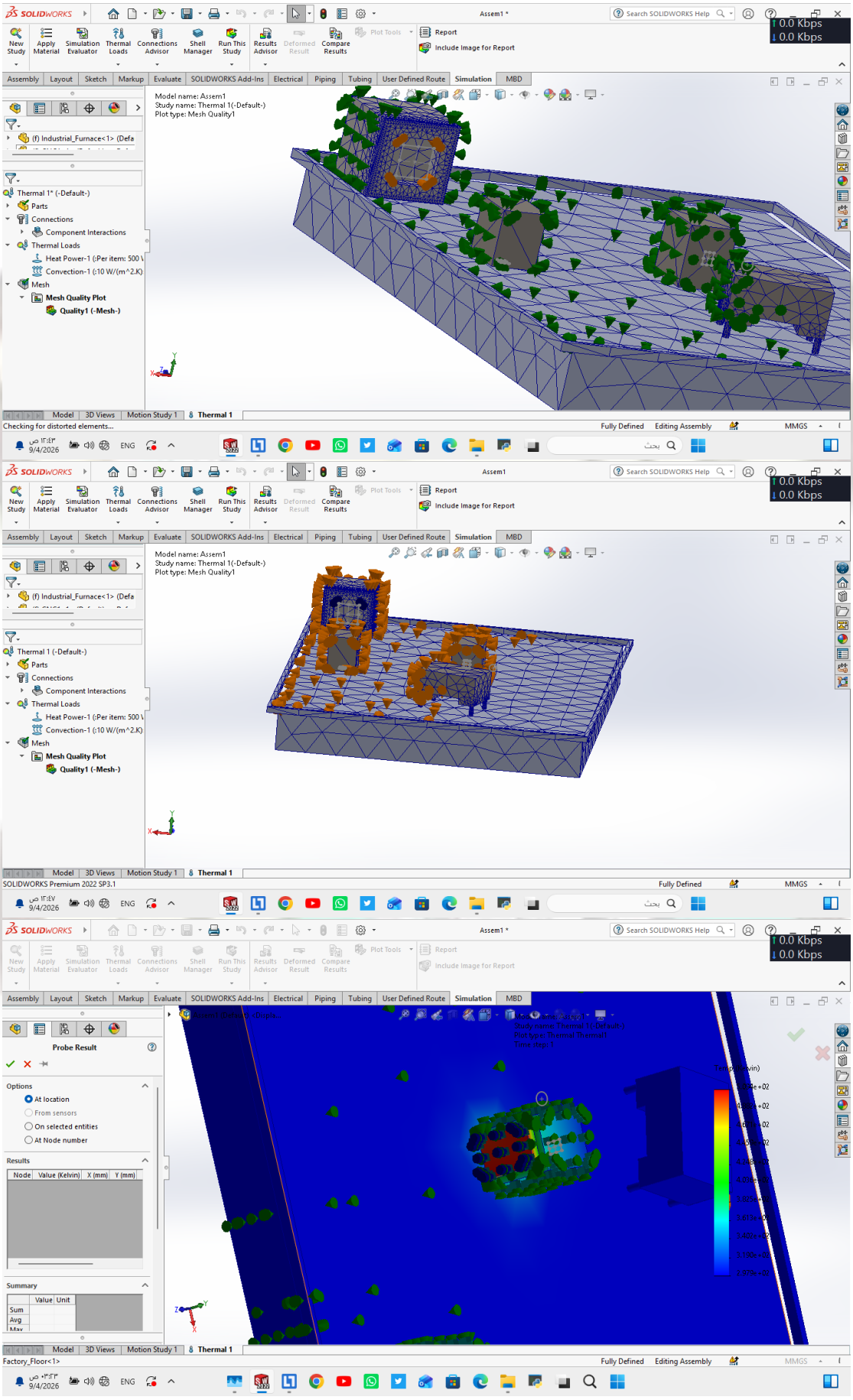

SOLIDWORKS
Factory_Floor
Search SOLIDWORKS Help
Extruded Boss/Base
Revolved Boss/Base
Swept Boss/Base
Lofted Boss/Base
Boundary Boss/Base
Extruded Cut
Hole Wizard
Revolved Cut
Swept Cut
Lofted Cut
Boundary Cut
Fillet
Linear Pattern
Rib
Draft
Shell
Wrap
Intersect
Mirror
Reference Geometry
Curves
Instant3D
0.0 Kbps
0.0 Kbps
Features
Sketch
Sheet Metal
Weldments
Markup
Evaluate
MBD Dimensions
SOLIDWORKS Add-Ins
MBD
Factory_Floor (Default) <<Default>_Di
History
Sensors
Annotations
Solid Bodies(1)
Material <not specified>
Front Plane
Top Plane
Right Plane
Origin
Boss-Extrude1
*Trimetric
Model
3D Views
Motion Study 1
SOLIDWORKS Premium 2022 SP3.1
MMGS
ENG
9/5/2026


SOLIDWORKS
Industrial_Furnace
Search SOLIDWORKS Help
Extruded Boss/Base
Revolved Boss/Base
Swept Boss/Base
Lofted Boss/Base
Boundary Boss/Base
Extruded Cut
Hole Wizard
Revolved Cut
Swept Cut
Lofted Cut
Boundary Cut
Fillet
Linear Pattern
Rib
Draft
Shell
Wrap
Intersect
Mirror
Reference Geometry
Curves
Instant3D
0.0 Kbps
0.0 Kbps
Features
Sketch
Sheet Metal
Weldments
Markup
Evaluate
MBD Dimensions
SOLIDWORKS Add-Ins
MBD
Industrial_Furnace (Default) <<Default
History
Sensors
Annotations
Solid Bodies(1)
Stainless Steel (ferritic)
Front Plane
Top Plane
Right Plane
Origin
Boss-Extrude1
(-) Sketch8
Cut-Extrude9
Sketch13
Fillet2
Front Plane
Model
3D Views
Motion Study 1
SOLIDWORKS Premium 2022 SP3.1
Editing Part
MMGS
ENG
9/5/2026


Industrial_Furnace
Front Plane
Model
3D Views
Motion Study 1
SOLIDWORKS Premium 2022 SP3.1
Editing Part
MMGS


WORK- CEII
Model
3D Views
Motion Study 1
Editing Part
MMGS


Assem1
3DEXPERIENCE Marketplace
Extend SOLIDWORKS with the 3DEXPERIENCE Platform
Connect to the 3DEXPERIENCE® platform and leverage its collaboration, product data and lifecycle management, organic shape design, simulation solutions, and more. (Subscription required.)
Use 3DEXPERIENCE Marketplace to source 3D content and services worldwide. (No license required.)
Let's get started
*Isometric
Model
3D Views
Motion Study 1
SOLIDWORKS Premium 2022 SP3.1
Fully Defined
Editing Assembly
MMGS


SOLIDWORKS
Assem1 *
Search SOLIDWORKS Help
0.0 Kbps
0.0 Kbps
New Study
Apply Material
Simulation Evaluator
Thermal Loads
Connections Advisor
Shell Manager
Run This Study
Results Advisor
Deformed Result
Compare Results
Plot Tools
Report
Include Image for Report
Assembly
Layout
Sketch
Markup
Evaluate
SOLIDWORKS Add-Ins
Electrical
Piping
Tubing
User Defined Route
Simulation
MBD
Model name: Assem1
Study name: Thermal 1(-Default-)
Plot type: Mesh Quality1
(f) Industrial_Furnace<1> (Defa
Thermal 1* (-Default-)
Parts
Connections
Component Interactions
Thermal Loads
Heat Power-1 (:Per item: 500 \
Convection-1 (:10 W/(m^2.K)
Mesh
Mesh Quality Plot
Quality1 (-Mesh-)
Model
3D Views
Motion Study 1
Thermal 1
Checking for distorted elements...
Fully Defined
Editing Assembly
MMGS
9/4/2026
ENG
Assem1
Model name: Assem1
Study name: Thermal 1(-Default-)
Plot type: Mesh Quality1
(f) Industrial_Furnace<1> (Defa
Thermal 1 (-Default-)
Parts
Connections
Component Interactions
Thermal Loads
Heat Power-1 (:Per item: 500 \
Convection-1 (:10 W/(m^2.K)
Mesh
Mesh Quality Plot
Quality1 (-Mesh-)
Model
3D Views
Motion Study 1
Thermal 1
SOLIDWORKS Premium 2022 SP3.1
Fully Defined
MMGS
9/4/2026
ENG
Assem1 *
Search SOLIDWORKS Help
0.0 Kbps
0.0 Kbps
Assembly
Layout
Sketch
Markup
Evaluate
SOLIDWORKS Add-Ins
Electrical
Piping
Tubing
User Defined Route
Simulation
MBD
Probe Result
Options
At location
From sensors
On selected entities
At Node number
Results
Node
Value (Kelvin)
X (mm)
Y (mm)
Summary
Value
Unit
Sum
Avg
Max
Study name: Thermal 1(-Default-)
Plot type: Thermal Thermal1
Time step: 1
Model
3D Views
Motion Study 1
Thermal 1
Factory_Floor<1>
Fully Defined
Editing Assembly
MMGS
9/4/2026
ENG

7. C++ / OOP Concepts Used

The concepts below are described only where they are genuinely present in the final implementation, with the specific class or line of code that demonstrates each one.

- Encapsulation — private data members (e.g. Point3D's x/y/z, GraphNode's gCost/hCost/parent) exposed only through public getter/setter methods.
- Constructors — every value class (Point3D, Obstacle, ThermalNode, GraphNode) provides both a default constructor and a parameterized constructor.
- Virtual destructors — CostFunction declares virtual ~CostFunction() = default;, the correct idiom for a polymorphic base class, so derived objects are destroyed safely through a base-class pointer.
- Pass-by-const-reference — nearly every function accepting a Point3D, GraphNode, or Environment takes it as a const reference rather than by value, avoiding unnecessary copies throughout the graph and search code.
- Operator overloading — Point3D overloads ==, !=, +, - and the stream insertion operator <<, so points can be compared, combined, and printed using natural syntax.
- Abstract classes and inheritance — CostFunction is an abstract base class with a pure virtual calculate() method; DistanceCost and ThermalCost both publicly inherit from it and provide their own implementation.
- Runtime polymorphism — AStar stores its active cost function as std::unique_ptr<CostFunction> and calls costFunction->calculate(...) without knowing at compile time whether it holds a DistanceCost or a ThermalCost — this is what lets setCostFunction(...) change the planner's entire behavior with no change to AStar::findPath itself.
- RAII / smart pointers — the active cost function is owned through a std::unique_ptr, so it is freed automatically; there is no manual delete anywhere in its lifecycle.
- Exception handling — ThermalDataReader::loadFromCSV wraps each row's parsing in a try { ... } catch (const std::exception&) block, so one malformed line in a 673-row dataset is reported instead of crashing the program.
- STL containers and algorithms — std::vector is used throughout (nodes, neighbors, obstacles, the A* open/closed lists); <algorithm> provides std::min_element (lowest-cost open node), std::find (list membership checks), and std::reverse (path reconstruction).

- File I/O — <fstream> (ifstream/ofstream) is used for all CSV reading and writing.
- Static utility methods — CSVReader::readThermalMap and CSVWriter::writePath are declared static, reflecting that they are stateless operations rather than behavior tied to one object instance.

For accuracy, it is worth being equally explicit about what the current implementation does not use. The A* open list is a std::vector scanned with std::min_element rather than a std::priority_queue, and the closed list is a linear vector rather than a hash set; for the dataset used here (673 nodes) this has no visible effect on performance, but a priority-queue-based version would scale better (see Section 11). Templates are used extensively via the Standard Library (vector<T>, unique_ptr<T>), but the project does not currently define any custom template classes or functions of its own, and path reconstruction is implemented iteratively rather than recursively.

8. Explanation of the Most Important Parts of the Implementation

8.1 Point3D and the Distance Metric

Every position in the system is ultimately a 3D coordinate. Point3D keeps this in one place: distanceTo() implements the Euclidean distance formula that both DistanceCost and the A* heuristic rely on, and the overloaded << operator is what allows the program to print lines such as “Start: (1.09, 11.08, 5.00)” directly.

8.2 Building the Graph from Irregular Simulation Data

A CAD thermal simulation does not produce a perfectly spaced grid the way a hand-authored test map would. Graph::getNeighbors() addresses this by, for every candidate node, determining which of six directions (+X, −X, +Y, −Y, +Z, −Z) it lies in relative to the current node, and keeping only the single closest candidate in each direction, provided it falls within a maximum connection distance. This lets the graph self-assemble correct six-way connectivity even though the real mesh spacing varies slightly across the imported dataset (see Section 10 for how the correct distance threshold was determined empirically).

8.3 The CostFunction Hierarchy

This is the architectural core of the “thermal-aware” idea. CostFunction defines one pure virtual method, calculate(current, next). DistanceCost returns the Euclidean

distance between the two nodes. ThermalCost returns zero if the destination node's temperature is at or below a configurable safeTemperature, and otherwise returns thermalWeight × (temperature − safeTemperature) — so cost increases smoothly once a node crosses the safety threshold, rather than the node being treated as a hard obstacle. Because AStar only ever calls the cost function through the base-class interface, main.cpp can select the desired planning behavior with a single call to setCostFunction.

8.4 The A* Search Loop

AStar::findPath follows the standard A* structure: it maintains an open list (nodes discovered but not yet expanded) and a closed list (nodes already expanded), repeatedly selecting the open node with the lowest f = g + h cost, expanding its neighbors, and updating their g/h costs and parent pointer whenever a cheaper route to them is found. When the goal node is popped from the open list, reconstructPath() walks backward through each node's parent pointer and reverses the result to produce the final path from start to goal.

8.5 Reading Real SolidWorks Thermal Data

ThermalDataReader::loadFromCSV is the bridge between the CAD/simulation world and the C++ engine. It skips the CSV header, then for each row reads the node ID and the X/Y/Z position in meters, followed by the temperature in Kelvin, which is converted to Celsius before constructing a ThermalNode. Parsing is wrapped in a try/catch block so that a single corrupted row is reported without terminating the whole load.

8.6 The MATLAB Application's Callback Flow

PlanOptimalPathButtonPushed is the most involved callback in the MATLAB application: it reads the six coordinate fields and two weight fields from the UI, validates them (rejecting negative weights, NaN coordinates, and out-of-range points against the loaded map's bounds), converts the physical X/Y coordinates into row/column indices via linear interpolation over the map's dimensions, times the call to localAStar with tic/toc, and — inside a try/catch block, mirroring the exception-safety approach used in the C++ engine's ThermalDataReader — reports any failure through a uialert dialog instead of crashing the app. On success, it overlays the returned path as a red line on the same axes as the thermal map, marks the start and goal with green and magenta markers, and writes the distance, thermal cost, total cost, node count, and elapsed time into the results panel.

9. Screenshots and Sample Outputs

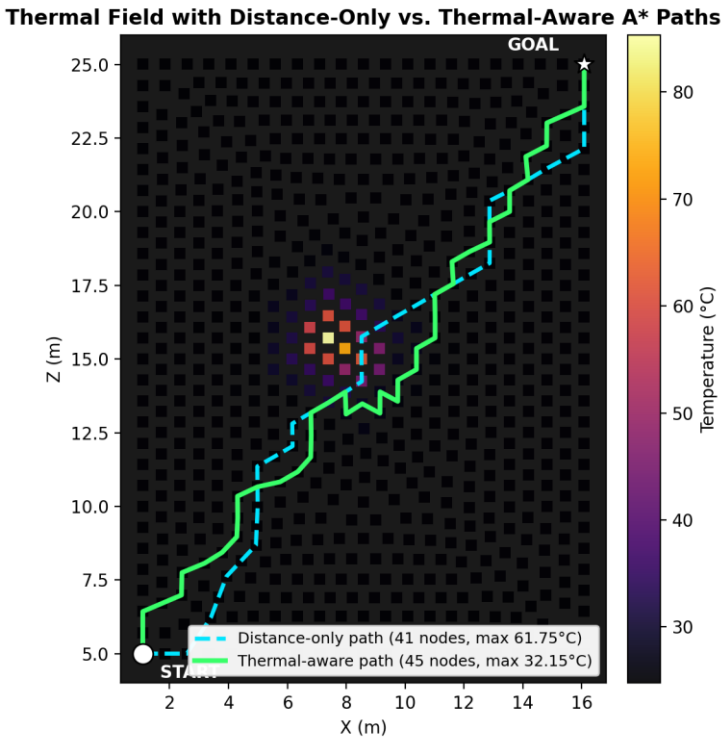


Figure 2. Real simulation data: the distance-only path cuts through an 85°C hot zone, while the thermal-aware path detours around it.

The table below summarizes the two runs shown above, both computed on the same real 673-node dataset between the same start and goal points:

Strategy	Path length (nodes)	Peak temperature on path	Result
DistanceCost (shortest path)	41	61.75 °C	Cuts through the hot zone
ThermalCost (thermal-aware)	45	32.15 °C	Detours around the hot zone

Below is an excerpt of the program's actual console output from a real run (steps 8–41 omitted for brevity; the full 45-step listing is produced identically on every run):

```
bash
$ ./thermal_planner
========================================
         THERMAL PATH PLANNER
========================================

Loading thermal map...
Thermal data loaded successfully.
Number of thermal nodes: 673
Thermal map loaded successfully.
Number of thermal nodes: 673

Building navigation graph...
Graph built successfully.

Start: (1.08833, 11.0781, 5.0031)
Goal:  (16.0883, 11.0781, 25.0031)

Searching for thermal-aware path...

Path found successfully!

Path nodes: 45

----------------------------------------
Step 0 : (1.08833, 11.0781, 5.0031) | Temperature = 24.95 C
Step 1 : (1.08833, 11.0781, 5.71738) | Temperature = 24.95 C
Step 2 : (1.08833, 11.0781, 6.43167) | Temperature = 24.95 C
Step 3 : (1.74542, 11.0781, 6.71119) | Temperature = 24.95 C
Step 4 : (2.40251, 11.0781, 6.99071) | Temperature = 24.95 C
Step 5 : (2.41931, 11.0781, 7.75195) | Temperature = 24.95 C
Step 6 : (3.18556, 11.0781, 8.06931) | Temperature = 24.95 C
Step 7 : (3.78476, 11.0781, 8.43837) | Temperature = 25.05 C
Step 8 : (4.28358, 11.0781, 8.96865) | Temperature = 25.05 C
Step 9 : (4.31284, 11.0781, 9.61134) | Temperature = 25.05 C
Step 10 : (4.30643, 11.0781, 10.3444) | Temperature = 25.05 C
Step 11 : (4.98542, 11.0781, 10.6636) | Temperature = 25.05 C
Step 12 : (5.75569, 11.0781, 10.8328) | Temperature = 25.05 C
Step 13 : (6.35524, 11.0781, 11.177) | Temperature = 25.05 C
Step 14 : (6.79521, 11.0781, 11.6709) | Temperature = 25.05 C
Step 15 : (6.81087, 11.0781, 12.4126) | Temperature = 25.25 C
Step 16 : (6.799, 11.0781, 13.1777) | Temperature = 25.35 C
Step 17 : (7.41675, 11.0781, 13.5314) | Temperature = 30.55 C
Step 18 : (7.96865, 11.0781, 13.8824) | Temperature = 32.15 C
Step 19 : (7.99487, 11.0781, 13.132) | Temperature = 26.95 C
Step 20 : (8.54677, 11.0781, 13.483) | Temperature = 28.45 C
Step 21 : (9.14784, 11.0781, 13.1628) | Temperature = 26.85 C
Step 22 : (9.15086, 11.0781, 13.8859) | Temperature = 27.45 C
Step 23 : (9.75193, 11.0781, 13.5657) | Temperature = 25.75 C
Step 24 : (9.75496, 11.0781, 14.2888) | Temperature = 26.45 C
Step 25 : (10.3883, 11.0781, 14.646) | Temperature = 25.95 C
Step 26 : (10.3883, 11.0781, 15.3602) | Temperature = 27.75 C
Step 27 : (11.0216, 11.0781, 15.7174) | Temperature = 25.35 C
Step 28 : (11.0216, 11.0781, 16.4317) | Temperature = 25.25 C
Step 29 : (11.0078, 11.0781, 17.1959) | Temperature = 25.15 C
Step 30 : (11.6411, 11.0781, 17.5531) | Temperature = 25.05 C
Step 31 : (11.5952, 11.0781, 18.3043) | Temperature = 25.05 C
Step 32 : (12.1965, 11.0781, 18.6484) | Temperature = 25.05 C
Step 33 : (12.8757, 11.0781, 18.9686) | Temperature = 25.05 C
Step 34 : (12.8707, 11.0781, 19.6578) | Temperature = 25.05 C
Step 35 : (13.555, 11.0781, 20.0031) | Temperature = 25.05 C
Step 36 : (13.555, 11.0781, 20.7174) | Temperature = 25.05 C
Step 37 : (14.1794, 11.0781, 21.0887) | Temperature = 25.05 C
Step 38 : (14.1045, 11.0781, 21.8722) | Temperature = 25.15 C
Step 39 : (14.812, 11.0781, 22.2372) | Temperature = 25.05 C
Step 40 : (14.8202, 11.0781, 23.0144) | Temperature = 25.05 C
Step 41 : (15.4542, 11.0781, 23.2945) | Temperature = 25.05 C
Step 42 : (16.0883, 11.0781, 23.5745) | Temperature = 25.05 C
Step 43 : (16.0883, 11.0781, 24.2888) | Temperature = 25.05 C
Step 44 : (16.0883, 11.0781, 25.0031) | Temperature = 25.05 C
----------------------------------------

Path exported successfully to:
data/path_result.csv

========================================
            PROGRAM FINISHED
========================================
$
```

The MATLAB application provides a second, interactive way to see the same idea in action. Figure 3 below is captured directly from the saved application — the actual startup screen, before a map has been loaded or a path planned:

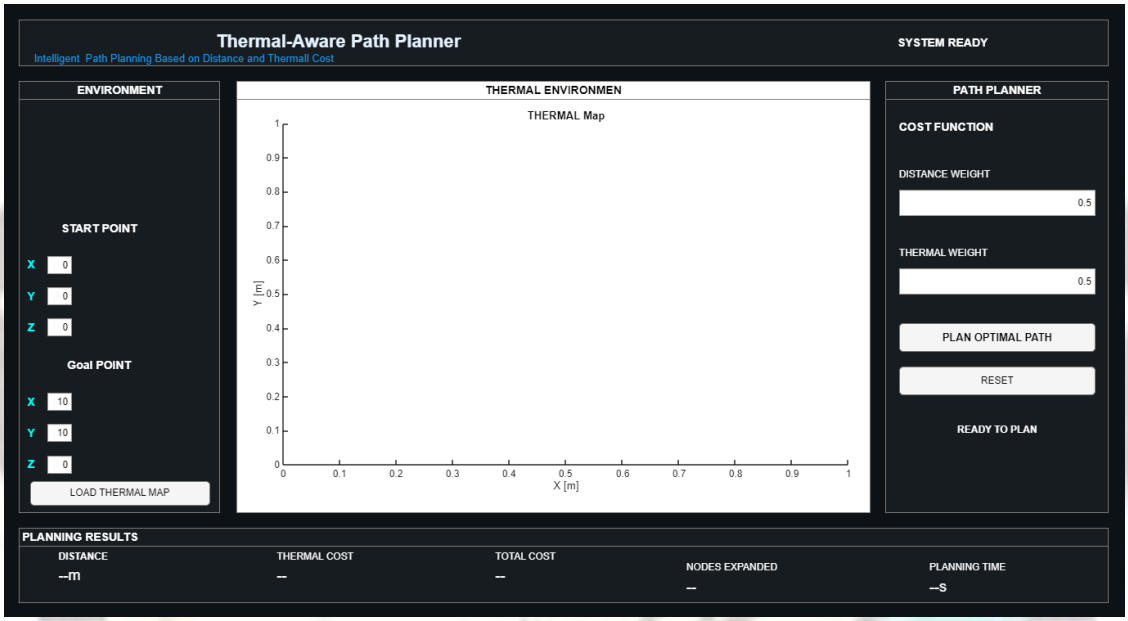
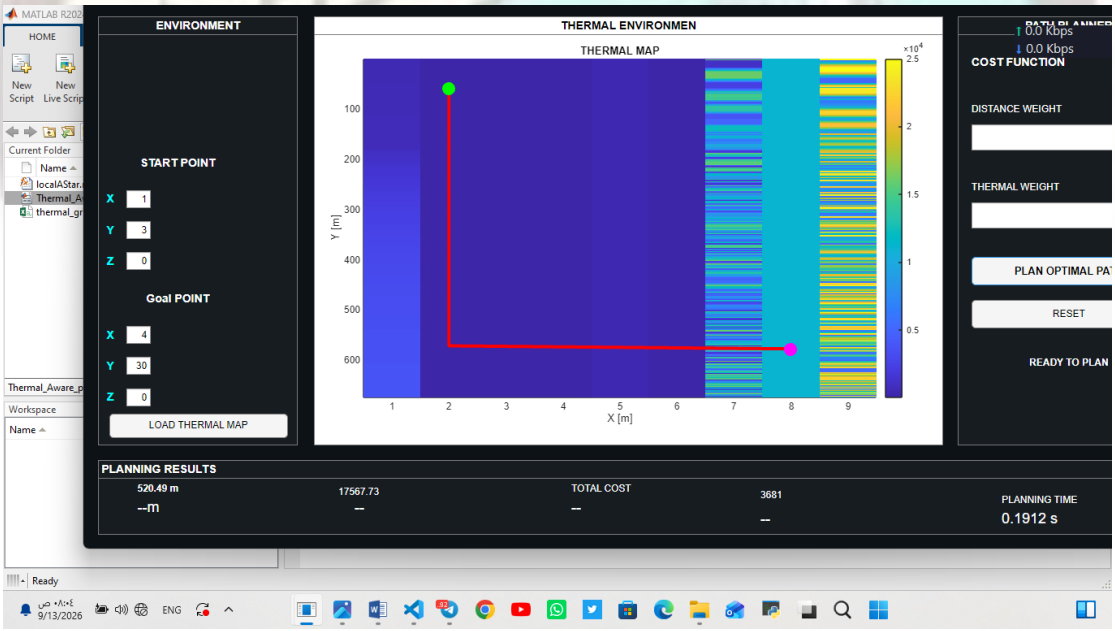


Figure 3. The MATLAB App Designer interface at startup: Environment panel (left), thermal map display (center), and Path Planner panel with live results bar (right/bottom).



10. Challenges Faced and How They Were Solved

The individual classes each compiled and behaved correctly in isolation, but the first full end-to-end run produced “No valid path found” for every start/goal pair. Three separate, stacked issues were identified and fixed through direct empirical testing — writing small diagnostic programs to print intermediate values, rather than relying on code inspection alone.

Issue	Symptom	Root cause and fix
CSV column order mismatch	Every position and temperature value was read incorrectly	ThermalDataReader::loadFromCSV assumed the column order (Node, Temperature, X, Y, Z), but the real SolidWorks export uses (Node, X, Y, Z, Temperature_K, Temperature_C, X_mm, Y_mm, Z_mm). Fixed by reordering the reads to match the true header and converting Kelvin to Celsius.
Neighbor-distance tolerance too small	Every node had zero neighbors, so the graph was fully disconnected	Graph::getNeighbors used a 0.01-unit tolerance, but the real mesh spacing in the dataset measured approximately 0.714 units. Fixed by raising the tolerance to 1.0, comfortably above the true spacing and below the next mesh shell (~1.43 units).
Missing output directory	Path export failed silently on a clean checkout	CSVWriter wrote to data/path_result.csv, but std::ofstream does not create missing directories. Fixed by calling std::filesystem::create_directories() on the destination path before opening the file.

Each fix was verified empirically rather than assumed correct from the code alone: a small diagnostic program printed the raw position and temperature values ThermalDataReader actually produced and compared them against the known coordinates of the intended start node; the true nearest-neighbor spacing in the dataset was measured directly across all 673 points; and the program was re-run from a completely clean checkout, with no pre-existing data/ folder, to confirm the directory fix. After all three fixes were applied together, the program successfully found a 45-node thermal-aware path on the first run.

11. Future Work

- Unify the two implementations — have the MATLAB application load the C++ engine's actual SolidWorks-derived CSV directly (or have the C++ engine additionally export a simple 2D grid the app already understands), so the interactive visualization reflects the exact path the graded C++ engine computed, rather than two independent cost models that could disagree on the same map.
- Align MATLAB cost reporting with its search objective — update calculatePathCost to accumulate the same min-max normalized temperature that localAStar actually searches on, so the displayed Thermal Cost and Total Cost are on the same scale as the quantity being optimized.
- Physical robot deployment — porting the planner onto a real mobile platform (e.g. Raspberry Pi, motor drivers, LiDAR or ultrasonic ranging, and live temperature sensing) so the thermal field is measured in real time rather than pre-computed in SolidWorks.
- Algorithmic scalability — replacing the current vector-based open list with a std::priority_queue binary heap, and the linear closed-list scan with a hash set, to keep the search efficient on maps much larger than the current 673-node dataset.
- Adaptive neighbor-distance detection — computing the graph's connection-distance threshold automatically from the dataset's own measured spacing, instead of a fixed constant, so the engine adapts to meshes of different densities without manual tuning.
- A meaningfully placed demonstration obstacle — the current obstacle in main.cpp does not intersect the real dataset's coordinates; a future iteration could position one so that obstacle avoidance is visibly demonstrated alongside thermal avoidance in the same run.

12. Conclusion

The Thermal-Aware Path Planner demonstrates that a small, well-factored addition to a classic A* implementation — a swappable, polymorphic cost function — is enough to fundamentally change what “optimal” means for a path, without touching the search algorithm itself. Using a real thermal dataset exported from a SolidWorks simulation, the completed C++ engine finds a 45-node path that keeps peak thermal exposure at 32.15°C, compared to 61.75°C for the shortest available route of 41 nodes — a substantial improvement in thermal safety for roughly a 10% increase in path length.

Beyond the algorithm itself, building the C++ engine surfaced and resolved three genuine integration issues between the SolidWorks export format and the ingestion code, which is arguably as representative of real engineering work as the algorithm design. Alongside it, the MATLAB application delivers exactly the interactive, visual front-end the project set out to provide, with its own independent implementation of the same thermal-aware idea on a 2D grid. The result is two working, complementary deliverables — a verified C++ computational engine and a functioning graphical planning tool — with a clear, honestly-scoped path toward unifying them and extending the system to a physically deployed robot.